Liquid Crypt4GH: Java Crypt4GH file system implementation

Dmitry Repchevsky[1,2], Salvador Capella-Gutierrez[1,2], Josep Lluis Gelpí[1,2,3]

[1] Barcelona Supercomputing Center (BSC)
[2] Spanish National Bioinformatics Institute (INB/ELIXIR-ES)
[3] Dept Biochemistry and Molecular Biomedicine, University of Barcelona (UB)

ELIXIR All Hands, 2-5 June 2025, Thessaloniki Greece

Genome analysis has become a regular procedure for disease diagnosis and research. Software frameworks and platforms employed for such analysis are quite complex and usually include sophisticated workflows executed in a cloud environment. Given the intrinsically sensitive nature of the processed data, providing solid security during the entire data lifetime is very important to enable broad adoption of the platforms. The Global Alliance for Genomics and Health (GA4GH) has developed the Crypt4GH file encryption standard, which despite being quite simple, provides a very powerful way to protect sensitive data.The Liquid Crypt4GH Java tool provides a transparent ("liquid") Crypt4GH support for Java-based tools that otherwise lacks Crypt4GH cryptography support.

Crypt4GH File Format

The format of Crypt4GH files is specified in the GA4GH File Encryption Standard. The Header of a Crypt4GH file contains a private key that can be encrypted for different users. This key is used to encrypt the file content, which is divided into 64K blocks and encrypted with the CHACHA20-Poly1305 algorithm.

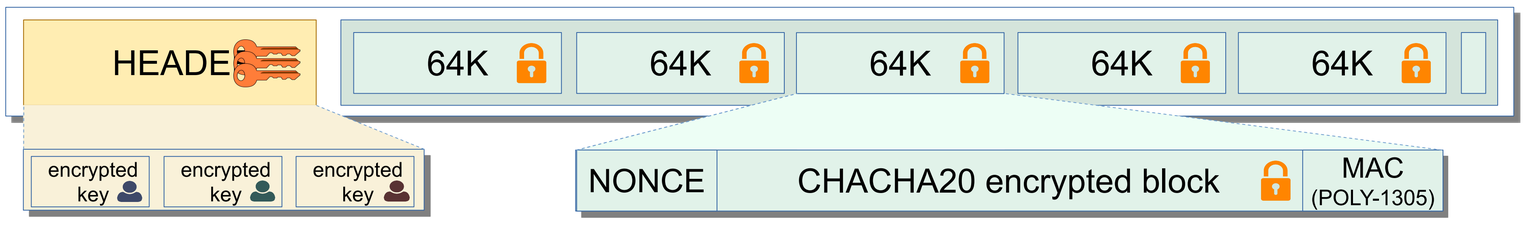

Fig. 1 Crypt4GH file format

Liquid Crypt4GH programming model

The primary advantage of the Liquid Crypt4GH library is that developers can use standard Java NIO API to work with Crypt4GH files without needing specific knowledge in cryptography (fig 2). In fully transparent mode, when a public key is provided, all written files are encrypted for that user. Alternatively, users can specify which files require encryption by using the "c4gh://" URI Scheme or use the Liquid Crypt4GH API directly (fig 3).

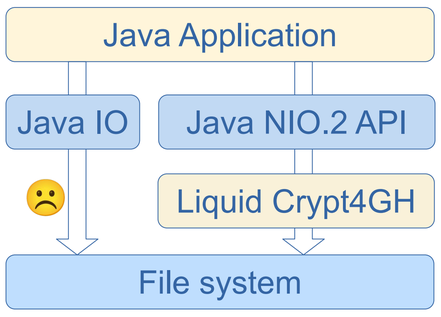

Fig. 2 Liquid Crypt4GH transparent encoding

```
Path unencrypted = Paths.get("/home/redmitry/unencrypted.bam");
Path encrypted = Paths.get("/home/redmitry/encrypted.bam.c4gh");

// the simplest way to encrypt a file - using standard java file copy function
Files.copy(unencrypted, encrypted, StandardCopyOption.REPLACE_EXISTING);

// byte[] data = Files.readAllBytes(encrypted);

// try (SeekableByteChannel out = Files.newByteChannel(encrypted,
//               StandardOpenOption.READ, StandardOpenOption.WRITE)) {
//         out.write(ByteBuffer.wrap(data));
// }
```
(a)

```
Crypt4ghKeys keys = new Crypt4ghKeys(privateKey);

Path unencrypted = Paths.get("/home/redmitry/unencrypted.bam");
Path encrypted = Paths.get("/home/redmitry/encrypted.bam.c4gh");

try (Crypt4ghSeekableByteChannel in =
               new Crypt4ghSeekableByteChannel(keys, unencrypted, StandardOpenOption.READ),
     Crypt4ghSeekableByteChannel out = new Crypt4ghSeekableByteChannel(keys, encrypted,
               Set.of(StandardOpenOption.CREATE, StandardOpenOption.READ,
                      StandardOpenOption.WRITE))))) {
    byte[] data = new byte[in.size()];
    in.read(ByteBuffer.wrap(data));
    out.write(ByteBuffer.wrap(data));
}
```
(b)

Fig. 3 Transparent (a) vs Direct (b) programming examples

Liquid Crypt4GH implementation considerations

- **Key type**: To store a private key in the header, Crypt4GH uses X25519 Elliptic-curve Diffie–Hellman (ECDH) encryption. Together with the proprietary Crypt4GH key format, Liquid Crypt4GH supports PKCS#8 Ed25519 keys. Ed25519 public keys are generated from the SHA 256/512 private key's hash, which means that Ed25519 (EdDSA) keys may be converted to X25519, but not the other way around. **The recommendation is to use ed25519 keys and generate x25519 if needed.**

- **Security Concerns:** Liquid Crypt4GH has no 64K limit, but still encrypts CHACHA20 blocks of 64 bytes while Java software may use smaller chunks for writing. Software should not update already distributed files. **In order to prevent security issues, avoid distributing different variants of encrypted files**.

- **Memory Mapping:** NIO.2 API supports *memory mapped files* via the MappedByteBuffer. This functionality is emulated via the *direct* ByteBuffer backed by MemorySegments. Memory segments keep Crypt4GH 64K cached blocks and may back more than one buffer. Writing to such buffers is not immediately reflected in the file system, but after the buffer is garbage collected or the *force()* method is called.
  This functionality is only available starting from Java 22. **In order to ensure proper library use, applications should avoid using outdated java.io API.**

Liquid Crypt4GH functionality

- Fully transparent proxy file system via property:
  `-Djava.nio.file.spi.DefaultFileSystemProvider=es.bsc.inb.ga4gh.jcrypt4gh.fs.Crypt4ghFileSystemProvider`
  In this mode Crypt4GH files are transparently encrypted and decrypted through the java.nio API.

- Custom Crypt4GH file system:
  `META-INF/services/java.nio.file.spi.FileSystemProvider ⏎`
  `es.bsc.inb.ga4gh.jcrypt4gh.fs.Crypt4ghFileSystemProvider`
  Developers still use standard Java NIO API, but explicitly specify which files should be encrypted / decrypted via "c4gh://" URI scheme.

- Direct Liquid Crypt4GH library usage:
  Developers can directly use the library to decrypt / encrypt files via the API.

- Use Liquid Crypt4GH as a command line tool:
  `java -jar liquid-crypt4gh.jar --encrypt myfile.txt -sk alice.p8`
  The command-line tool provides Crypt4GH, PKCS#8 / X.509 keys generation / conversion as well as encryption / decryption operations.

Dmitry Repchevsky
Research Software Engineer
dmitry.repchevski@bsc.es

Barcelona Supercomputing Center
Centro Nacional de Supercomputación

European Genomic Data Infrastructure

The GDI project is co-funded by the European Union's Digital Europe Programme under grant agreement number 101081813

ELIXIR ALL HANDS 2025